Sovereign On-Premise Agentic AI Workbench

Operator Guide: Dashboard, Query Pipeline,
Reindexing, and Grounding Verification

SIH26117 | Mangalore Refinery and Petrochemicals Limited

10 September 2026

Contents

1 Dashboard Field Guide

The operator console runs at `http://127.0.0.1:8501` (Streamlit) and talks to the API at `http://127.0.0.1:8077` (FastAPI) over loopback HTTP. As of 10 September 2026 the console holds *no* state of its own – it opens neither the graph store nor the vector index directly, because the embedded Kużu graph database does not permit a read-only handle and a read-write handle on the same path at once (confirmed directly from the installed `kuzu==0.11.3` library). Every field described below is either read from the API or computed locally from the filesystem; the "backing call" column states which.

1.1 Sidebar

Field	Meaning	Backing call
External calls	Running count of outbound connections seen from the API process's own tree plus the Ollama server – not the whole machine.	`GET /sovereignty`
Blocked	Count of deliberate canary attempts that were correctly refused.	`GET /sovereignty`
Sandbox	Isolation backend for code execution: `netns` (Linux network namespace) on this machine, `docker` if available, else `rlimit only`.	`GET /sovereignty`
Audit chain	**valid** or **BROKEN at #N** – every audited event is hash-chained to the previous one; a break means a record was altered or removed.	`GET /sovereignty`
Attempt external call	Deliberately dials `1.1.1.1:53`. Success would mean the sandbox is *not* sovereign.	`POST /sovereignty/canary`
Graph nodes / edges	Live totals from the knowledge graph.	`GET /health`
Vectors	Chunk count in the embedded Qdrant vector store.	`GET /health`

1.2 Tab 1 — Ask

The main agentic entry point. **Goal** is free text; **Max iterations** (1–3) bounds the plan/execute/critique loop; **Max steps** (1–8) bounds tool calls per plan. **Run agent** calls `POST /ask` and renders:

- **Answer** – the final synthesised text, grounded only in what the tool calls actually returned (see §4).
- **Steps** – one expander per tool call: ✓/×for success, milliseconds taken, the resolved arguments *after* coercion, and the raw tool output. This is the most important panel for verifying the system is not fabricating: it is the evidence trail.
- **Model routing** – which model handled `plan` vs `reason` calls this run, and why.
- **Artefacts** – any generated file (`.docx`/`.xlsx`), downloadable and previewed inline.

1.3 Tab 2 — Search

Retrieval only, no tool-calling agent, near-instant. Calls `GET /search`. Shows which of the four retrieval modes fired (§2.2) and why, the graph nodes reached, community summaries if the query

is corpus-wide, and the raw matched passages with document/page provenance. Use this *before* trusting an Ask answer: if Search finds nothing relevant, Ask should refuse.

1.4 Tab 3 — Graph

Direct multi-hop traversal. Calls `GET /graph/neighborhood` with an entity name and hop depth (1–4). Reports the anchor node, everything reached within that many hops, and specifically flags entities reachable *only* beyond one hop – the case plain vector similarity cannot find (§2.4). Two expanders below it, **Active ontology** and **Node counts**, read static config and `GET /health` respectively – no live query.

1.5 Tab 4 — Documents

Pure filesystem view of `data/corpus/` (indexed vs. skipped by extension) and `data/artifacts/` (generated deliverables). The uploader copies a file into `data/corpus/` but does *not* index it automatically – see §3.

1.6 Tab 5 — Models

The declarative model catalogue from `config/models.yaml` (`GET /models`): id, Ollama tag, VRAM footprint, context length, priority, capabilities, and whether it is currently resident. Below it, a **Sandbox check** box runs a canned Python snippet through the same sandbox tool the agent uses, independent of the API, to prove code execution is genuinely isolated (rubric point 3).

1.7 Tab 6 — Audit

Tails the hash-chained audit log (`GET /audit`): every plan, tool call, canary attempt, and startup event, in order, with the chain-validity verdict at the top.

1.8 Live verification (10 September 2026)

Every backing call above was exercised directly against the running instance while writing this document:

```
GET /health -> {"ok": true, "graph_nodes": 9, "graph_edges": 3,
                    "vector_chunks": 4, "tools": 12}
GET /sovereignty -> {"external_connections": 0, "blocked_attempts": 0,
                    "sovereign": true, "audit_chain_valid": true}
GET /models -> 200, catalogue + resident + routing decisions present
GET /audit?n=5 -> 200, chain_valid: true, real plan/tool_call records
GET /search?q=... -> mode: "traverse", correct P-101A/E-204/V-1201 chain
GET /graph/neighborhood?entity=P-101A -> anchor + 2-hop neighbourhood, correct
POST /ask -> full plan -> execute -> synthesise cycle, correct
```

All seven endpoints returned ✓ 200 with correct content. The Streamlit process log showed no exception and no lock error across a full restart.

2 How a Query Gets Processed

Two independent routing decisions happen on every request: *which model* handles it, and *which retrieval mode* answers it. Neither is hardcoded per query type – both are computed from the request itself.

2.1 Model routing

`app/router/classifier.py` runs a two-stage classifier. Stage 1 is free: regular-expression heuristics match the prompt against seven task categories (`code, vision, extract, reason, plan, doc, summarize`) – an image attachment forces `vision`; code fences or words like *debug, refactor* force `code`; words like *approval note, docx, memo* force `doc`; and so on. Only when no heuristic fires confidently does stage 2 fall back to a small local classifier model. `app/router/manager.py` then picks the highest-priority model in `config/models.yaml` that declares the needed capability and fits the remaining VRAM budget (13.5 GB on this 16 GB card), evicting the least-recently-used resident model if it does not. Every decision – model chosen, reason, what was evicted – is recorded and shown in the Models tab and in each Ask response's **Model routing** panel.

2.2 Retrieval routing

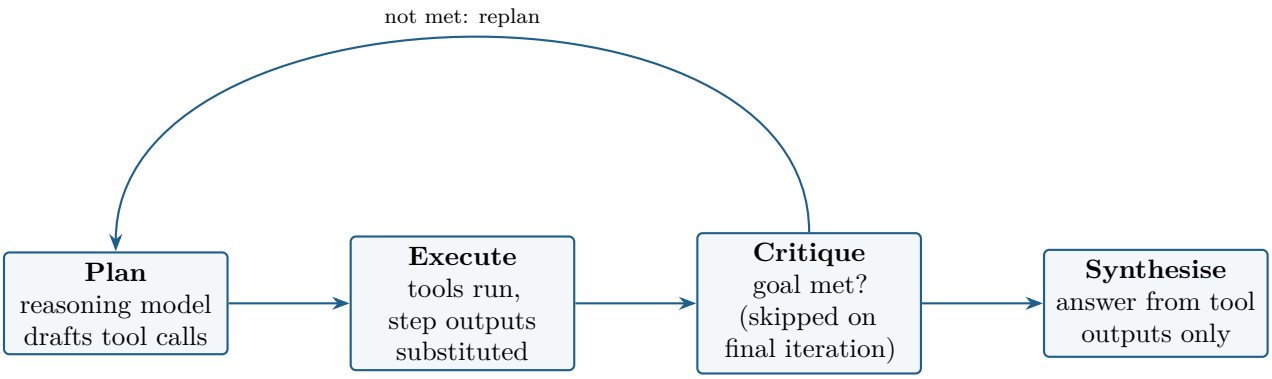
`app/graphrag/retrieve.py` classifies the query itself into one of four modes before anything is fetched:

Mode	Fires when
`traverse`	Impact/dependency language (*if we, affects, downstream, isolate, shutdown, ...*) *and* an explicit equipment-tag pattern (e.g. `P-101A`) are both present. Walks up to 3 hops in the graph.
`global`	Corpus-wide language (*overall, across, themes, how many, summarise all*). Uses community summaries instead of individual chunks.
`local`	An explicit tag with no impact language. 1-hop entity neighbourhood plus matching text chunks.
`vector`	No structural signal at all. Falls back to hybrid dense + BM25 vector search over text chunks.

This is why *"Which units are affected if P-101A is isolated"* correctly triggers graph traversal rather than plain text search: it matches both the impact regex and the tag regex.

2.3 The agent loop

`app/agent/loop.py` runs Plan → Execute → Critique → Synthesise, repeated up to **Max iterations** times:



The planner never sees tool results before writing the plan, so later steps reference earlier ones with a `{{stepN}}` placeholder, resolved at execution time (`Agent._resolve`). Every tool call, its coerced

arguments, and its raw output are hash-chained into the audit log – this is exactly what the Ask tab's Steps panel and the Audit tab display.

2.4 Worked example, traced live

Query: *"Which units are affected if P-101A is isolated, and draft an approval note recommending seal replacement."*

1. Retrieval router: tag `P-101A` + impact word *affected* ⇒ `traverse` mode, 3-hop walk.
2. `graph_query(entity="P-101A", hops=2)` returns `E-204` (1 hop, direct `FEEDS`) and `V-1201`.
3. `kb_search` pulls the grounding text: the isolation note and the P-101A inspection findings (seal weepage, recommend replacement).
4. The synthesis step composes the answer citing `[piping_topology_unit3 p.1]` and `[inspection_report_p101a p.1]` – every claim traces to a retrieved passage, not to model memory.

The critical property: `V-1201` is reachable from `P-101A` only by walking `FEEDS` edges through `E-204` in the graph – no embedding model retrieves that connection from text similarity alone. This is the Graph RAG differentiator this project adds beyond the problem statement's literal requirements.

3 Adding New Data: The Correct Startup Procedure

3.1 Why order matters

Kużu (the graph database) permits *either* many read-only handles on a database path, *or* exactly one read-write handle – never a mix, and never two read-write handles. The API process (`app/main.py`) is the only process in this system that opens `data/stores/graph` in read-write mode. `make index` (`scripts/02_build_index.py`) also opens it read-write, to extract and write graph nodes/edges from new documents. **The two can never run at the same time.** The Streamlit console does not open the store at all as of 10 September 2026, so it does not need to be stopped for this – only the API does.

3.2 Procedure

1. **Check ports and locks are clear** before starting or stopping anything:

```
ss -ltn | grep -E ':8077|:8501' # both should be free before a fresh start
fuser -v data/stores/graph # should be empty before reindexing
```

2. **Stop the API** if it is running (Ctrl-C in its terminal, or):

```
pkill -f "uvicorn app.main:app"
```

   The Streamlit console may keep running through this step – it will simply show stale numbers until you restart it, and its next call to the API will fail cleanly with "cannot reach the API" rather than a lock error.

3. **Add the new documents.** Copy `.pdf`, `.txt`, or `.md` files into `data/corpus/` (via the Documents tab uploader, or directly on disk). **Image files** (`.png`, `.jpg`) are *not* picked up by the indexer – they remain available to the vision tool on demand but are not chunked into the graph or vector store.

4. **Reindex:**

```
make index
# equivalent to: ./.venv/bin/python scripts/02_build_index.py
```

This is safe to re-run: extraction is cached by chunk hash, so only new or changed content costs LLM time. It ingests the corpus, adds chunks to the vector store, extracts graph nodes/edges (LLM-driven, the slow step), and re-summarises communities.

5. **Restart both services, with the port check built in:**

```
make start
# equivalent to: ./scripts/06_start_all.sh
```

This script refuses to start if port 8077 or 8501 is already occupied, confirms Ollama is up, starts the API, polls `/health` until it responds, and only then starts the console – eliminating the process-ordering problems this project hit twice during development.

6. **Verify the new data landed:**

```
curl -s http://127.0.0.1:8077/health
```

Check `graph_nodes`, `graph_edges`, and `vector_chunks` increased as expected, then confirm in the Documents tab that the new file shows `indexed: yes`.

3.3 Quick reference

Command	Purpose
`make check`	Verify the environment (Python, models pulled, GPU)
`make index`	(Re)build vector index + knowledge graph from `data/corpus/`
`make start`	Port-checked, correctly-sequenced launch of API + console
`make serve`	API alone, on 8077
`make app`	Console alone, on 8501 (requires the API already running)
`make test`	Run the 70-test suite
`fuser -v data/stores/graph`	See which process(es) hold the graph store open

4 Grounding: What Was Fixed, and How to Verify It Yourself

4.1 The failure that was observed

Asked to find a research paper on *"Attention Is All You Need"* (which does not exist in this corpus and cannot exist – this machine has no internet access), the agent: searched the knowledge base and found nothing relevant; tried three times to fall back to a general web search, which failed every time because the sandbox has no route out; and then, having no instruction to do otherwise, took the closest superficially-similar document actually in the corpus (an unrelated paper on transmission-line defect detection that happens to mention "attention mechanisms") and presented *that* as the answer, complete with a real but entirely unrelated DOI link.

4.2 Root cause

Two gaps in `app/agent/loop.py`:

1. The *planning* prompt never told the model that internet access does not exist, so the planner would draft web-search steps that were guaranteed to fail, wasting iterations before falling through to a bad answer.

2. The *synthesis* prompt said only "answer using the tool results below" – with no instruction covering the case where those results do not actually address the request. When the only material found was off-topic, the model filled the gap by presenting it as on-topic rather than saying so.

4.3 The fix

Both prompts were made explicit:

- **Planning prompt** now states: *"THIS MACHINE HAS NO INTERNET ACCESS, EVER ... if kb_search/graph_query cannot find what the request needs, the correct plan is a single step that reports the knowledge base has nothing relevant – not a retry against the internet."*

- **Synthesis prompt** now states: *"If the tool results do not actually address the request ... you MUST say plainly that it was not found in the knowledge base and there is no internet access to fetch it. Do NOT present an unrelated document as if it answers the request, and do NOT answer from your own training knowledge."*

Re-running the exact same query after the fix, live:

```
"answer": "The request for a research paper on \"Attention is All You
Need\" and its link was not found in the knowledge base. There is no
internet access to fetch it."
```

with zero web-search steps drafted at all.

4.4 Important caveat – what this fix is, and is not

This is a **prompt-level instruction**, enforced by the reasoning model following directions – it is not a mechanical gate that makes fabrication structurally impossible. A sufficiently different phrasing of an off-corpus request, or a weaker/differently-tuned model, could in principle still slip past it. The recommended hardening not yet implemented is a relevance-score threshold on retrieval that mechanically blocks synthesis when nothing scores above it, plus a post-hoc check that every claim in the answer traces to a retrieved chunk. Until that lands, treat every answer as needing the verification protocol below, not as unconditionally trustworthy.

4.5 How to check your own project is not hallucinating

1. **Cross-check every citation.** Every grounded answer cites `[doc_id p.N]`. Open the Documents tab, confirm that `doc_id` is actually in `data/corpus/` and actually says what the answer claims. A citation to a document that is not listed there, or that does not contain the claimed fact, is a fabrication.

2. **Run the same query in Search first.** Search is retrieval only – no synthesis, so it cannot fabricate. If Search returns nothing relevant, the corresponding Ask answer *must* be a refusal. If Ask instead gives a confident answer while Search found nothing, that is exactly the failure mode described above, recurring.

3. **Probe with a known-absent question.** Periodically ask something you know is not in the corpus (a piece of general trivia, or a real fact about equipment that does not exist in `data/corpus/`). The correct behaviour is an explicit refusal citing no internet access, every time.

4. **Read the Steps panel, not just the Answer.** Every tool call's raw output is shown in the Ask tab. If the final Answer states something the raw retrieved text does not support, that is fabrication even when retrieval itself found genuinely relevant material – a subtler failure than citing the wrong document entirely.

5. **Verify the audit chain.** The Audit tab's "Hash chain verified" status confirms the recorded trace has not been altered after the fact, so the Steps panel is a trustworthy record of what actually happened, not a reconstruction.